

aptana® Jaxer® The world's first Ajax server

Home Quick Start **Guide** Tutorials API Screencasts Forums Download

Jaxer Guide

- Introduction
- Setup
- Develop
- Debug

Working with Database Content

Getting Data: Jaxer.DB.ResultSet

To retrieve data from the database you execute an SQL SELECT query which returns a new instance of the Jaxer.DB.ResultSet class. The resultset contains all the data returned from the query, which means you can continue using it even after the connection is closed. The data in it also will not change even if the database data changes after you created the object.

Jaxer.DB.ResultSet offers multiple ways of getting at its data. A common one is through the rows property, which is an Array of objects, one per returned row. Each row is a simple JavaScript object with a property per column, the value being the value of the row's cell for that column:

```
1. var resultSet = Jaxer.DB.execute("SELECT * FROM
   ↳ booktest");
2. var firstTitle = resultSet.rows[0].title; // equals "Mary Poppins"
```

Contents

[Getting Data: Jaxer.DB.ResultSet](#)
[A Functional Approach](#)
[Moving Data into Forms](#)
[Form Validation](#)
[Writing to the DB](#)

Resource and API Links

[Jaxer.DB](#)
[Jaxer.DB.ResultSet](#)
[ActiveRecordJS.org](#)

Use rows.length to get a count of the returned rows. For each row, you may alternatively want all the cell values in one array, in the order returned by the query so each row has a property called \$values that holds an Array of its cells' values.

To get all the data as a rectangular array (that is, a table), the resultSet's rowsAsArrays property is an Array of the rows returned by the query but with each row an Array itself containing the values of the row's cells (the same as \$values, above).

```
1. var resultSet = Jaxer.DB.execute("SELECT * FROM booktest");
2. resultSet.rowsAsArrays.forEach(function(row) {
3.   print(row.join(" | ")); });
```

The firstRow/singleRow properties return an object containing the first row from a result set while lastRow conversely is object based on the final row of the result set. Jaxer.DB.ResultSet has matching singleRowAsArray and lastRowAsArray methods that convert the single row objects to an array with the added advantage that the methods take an array parameter which lets you filter the columns included in the return value.

```
1. var resultSet = Jaxer.DB.execute("SELECT * FROM booktest");
2. resultSet.singleRowAsArray([2, 3, 5, 8]).forEach(function(col) {
3.   print('Col value: '+col+'\n');
4. });
```

The singleResult property is your friend if you know a query returned a single result or you only care about the first cell in the first row; singleResult is always null if there is no data to return.

```
var numBooks = Jaxer.DB.execute("SELECT COUNT(*) FROM booktest").singleResult;
```

To determine if the resultSet has any data, or to distinguish a singleResult that's null because the database had a NULL in that cell versus returned no data from the query, use the hasData property.

```
1. var resultSet = Jaxer.DB.execute("SELECT * FROM booktest");
2. if(resultSet.hasData) {
3.   resultSet.rowsAsArrays.forEach(function(row) {
4.     print(row.join(" | ")); });
5. }
```

The resultSet's columns property contains an array of the names of the columns returned, in the order returned by the query. If you know a column's name, you can find its index using the resultSet's indexOfColumn() function.

Quick Printing

Jaxer.DB.ResultSet also gives you two convenience methods: toHTML and toString. The former returns the result set as an HTML table, the latter returns the result set as a string.

Avoiding SQL Injection Attacks

One of the biggest tar pits in web application development is avoiding the bad guys and hooligans. SQL Injection is the name for a nasty security hole to which web apps can be vulnerable if the developer doesn't take the appropriate precaution. Fortunately, Jaxer makes avoiding this attack very simple: whenever you pass a SQL statement to Jaxer.DB.execute (or any of the APIs which execute SQL on the database) put a ? (question mark) in your SQL where user-supplied input goes and pass the actual values as one or more parameters. Jaxer will sanitize the input value before passing the final statement to your database.

Examples

- Selects: Jaxer.DB.execute('SELECT * FROM Table1 WHERE fieldX = ? AND fieldY = ?', value1, value2)
- Updates: Jaxer.DB.execute('UPDATE Table1 SET field1 = ?, field2 = ?, field3 = ? WHERE keyValueField = ?', valA, valB, valC, valD)
- Inserts: Jaxer.DB.execute('INSERT INTO Table1 VALUES(?, ?, ?, ?)', val1, val2, val3, v4);

- Deletes: Jaxer.DB.execute('DELETE FROM Table1 WHERE keyValueField = ?', val6);

Alternative: Use the Google Gears DB API

The Jaxer.DB.ResultSet API currently has approximately 30 methods for accessing the data stored in the properties discussed here. These are convenience methods intended to simplify reading a Jaxer.DB.ResultSet. For example, say you prefer to use the Google Gears DB API; the Jaxer.DB.ResultSet also supports its access methods:

```
01. var resultset = Jaxer.DB.execute("SELECT * FROM booktest");
02. var numFields = resultset.fieldCount();
03. var header = [];
04. for (var i=0; i<numfields; i++) { header.push(resultset.fieldname(i)); } print(header.join("
05.     </div>
06.     </div>
07.
08.     <pre name="code" class="brush: js;">
09. // Results shown in Studio's Jaxer Shell:
10.
11. id | title
12. 1 | Mary Poppins
13. 2 | Robinson Crusoe
14. 3 | Harry Potter</pre>
15.
16.     <h2 id="functional">A Functional Approach</h2>
17.     <p>Jaxer.DB includes methods to apply against results in a functional approach as
18.     <ul>
19.         <li>every(Object fn) : Boolean</li>
20.         <li>filter(Object fn) : Array</li>
21.         <li>forEach(Object fn) : void </li>
22.         <li>map(Object fn) : Array</li>
23.         <li>reduce(Object fn, Object initialValue) : Object</li>
24.         <li>reduceRight(Object fn, Object initialValue) : Object</li>
25.         <li>some(Object fn) : Boolean</li>
26.     </ul><br/>
27.
28.     <h3>Accumulate the sum of IDs from our booktest table</h3>
29.     <pre name="code" class="brush: js;">var sumOfIds = conn.execute("select * from
30.     <pre>
31.         return previousValue + currentValue[0];
32.     </pre>
33.
34.     <h3>Double each ID</h3>
35.     <pre name="code" class="brush: js;">var doubleTheIds = conn.execute("select * from
36.     <pre>
37.         return 2 * row[0];
38.     </pre>
39.
40.     <h2 id="lookupExample">Moving Data into Forms</h2>
41.
42.     <p>
43.         One of the most common database design patterns is to store lookup data, sets of
44.         valid values for a given field, in the database. Jaxer makes it simple to
45.         retrieve these values and insert them into, say, select fields on your forms, as
46.         shown in the following example.
47.     </p>
48.     <div id="selectLookup">
49.         <div class="body">
50.             <textarea name="code" class="brush: html;">
51. <html>
52.     <body>
53.         <script runat="server-proxy">
54.             /* Requires a table Lookups with fields:
55.             * ID: Primary key
56.             * DataSet: Group name for a set of lookup values
57.             * luName: Text to display
58.             * luValue: Value to store (optional)
59.             */
60.             function getLookups(dSet, useLUV) {
61.                 var selectString = "";
62.                 var lookups = Jaxer.DB.execute("SELECT * FROM Lookups WHERE DataSet=?",dSet);
63.                 if(lookups.rows > 0) {
64.                     selectString += "<select id='dSet'+dSet+'>";
65.                     // Loop resultset to build select option list
66.                     lookups.forEach(function(curValue, idx, row) {
67.                         if (useLUV) {
68.                             selectString += "<option value='" + row.luValue + ">" + row.luName
69.                             <pre>
70.                             } else {
71.                                 selectString += "<option>" + row.luName + "</option>";
72.                             }
73.                         }
74.                     })
75.                     selectString += "</select>";
76.                 } else {
77.                     // error condition
78.                     alert('Could not find any lookups for dataset ' +dSet);
79.                 }
80.                 return selectString;
81.             }
82.         </script>
83.         <script>
84.             window.onload = function() {
85.                 var sel = document.getElementById('setStatus');
86.                 sel.innerHTML = getLookups('statuses', false);
87.             }
88.         </script>
89.     </div>
90.     </div>
91. </div>
92. </div>
93. </div>
94. </div>
95. </div>
96. </div>
97. </div>
98. </div>
99. </div>
100. </div>
101. </div>
102. </div>
103. </div>
104. </div>
105. </div>
106. </div>
107. </div>
108. </div>
109. </div>
110. </div>
111. </div>
112. </div>
113. </div>
114. </div>
115. </div>
116. </div>
117. </div>
118. </div>
119. </div>
120. </div>
121. </div>
122. </div>
123. </div>
124. </div>
125. </div>
126. </div>
127. </div>
128. </div>
129. </div>
130. </div>
131. </div>
132. </div>
133. </div>
134. </div>
135. </div>
136. </div>
137. </div>
138. </div>
139. </div>
140. </div>
141. </div>
142. </div>
143. </div>
144. </div>
145. </div>
146. </div>
147. </div>
148. </div>
149. </div>
150. </div>
151. </div>
152. </div>
153. </div>
154. </div>
155. </div>
156. </div>
157. </div>
158. </div>
159. </div>
160. </div>
161. </div>
162. </div>
163. </div>
164. </div>
165. </div>
166. </div>
167. </div>
168. </div>
169. </div>
170. </div>
171. </div>
172. </div>
173. </div>
174. </div>
175. </div>
176. </div>
177. </div>
178. </div>
179. </div>
180. </div>
181. </div>
182. </div>
183. </div>
184. </div>
185. </div>
186. </div>
187. </div>
188. </div>
189. </div>
190. </div>
191. </div>
192. </div>
193. </div>
194. </div>
195. </div>
196. </div>
197. </div>
198. </div>
199. </div>
200. </div>
201. </div>
202. </div>
203. </div>
204. </div>
205. </div>
206. </div>
207. </div>
208. </div>
209. </div>
210. </div>
211. </div>
212. </div>
213. </div>
214. </div>
215. </div>
216. </div>
217. </div>
218. </div>
219. </div>
220. </div>
221. </div>
222. </div>
223. </div>
224. </div>
225. </div>
226. </div>
227. </div>
228. </div>
229. </div>
230. </div>
231. </div>
232. </div>
233. </div>
234. </div>
235. </div>
236. </div>
237. </div>
238. </div>
239. </div>
240. </div>
241. </div>
242. </div>
243. </div>
244. </div>
245. </div>
246. </div>
247. </div>
248. </div>
249. </div>
250. </div>
251. </div>
252. </div>
253. </div>
254. </div>
255. </div>
256. </div>
257. </div>
258. </div>
259. </div>
260. </div>
261. </div>
262. </div>
263. </div>
264. </div>
265. </div>
266. </div>
267. </div>
268. </div>
269. </div>
270. </div>
271. </div>
272. </div>
273. </div>
274. </div>
275. </div>
276. </div>
277. </div>
278. </div>
279. </div>
280. </div>
281. </div>
282. </div>
283. </div>
284. </div>
285. </div>
286. </div>
287. </div>
288. </div>
289. </div>
290. </div>
291. </div>
292. </div>
293. </div>
294. </div>
295. </div>
296. </div>
297. </div>
298. </div>
299. </div>
300. </div>
301. </div>
302. </div>
303. </div>
304. </div>
305. </div>
306. </div>
307. </div>
308. </div>
309. </div>
310. </div>
311. </div>
312. </div>
313. </div>
314. </div>
315. </div>
316. </div>
317. </div>
318. </div>
319. </div>
320. </div>
321. </div>
322. </div>
323. </div>
324. </div>
325. </div>
326. </div>
327. </div>
328. </div>
329. </div>
330. </div>
331. </div>
332. </div>
333. </div>
334. </div>
335. </div>
336. </div>
337. </div>
338. </div>
339. </div>
340. </div>
341. </div>
342. </div>
343. </div>
344. </div>
345. </div>
346. </div>
347. </div>
348. </div>
349. </div>
350. </div>
351. </div>
352. </div>
353. </div>
354. </div>
355. </div>
356. </div>
357. </div>
358. </div>
359. </div>
360. </div>
361. </div>
362. </div>
363. </div>
364. </div>
365. </div>
366. </div>
367. </div>
368. </div>
369. </div>
370. </div>
371. </div>
372. </div>
373. </div>
374. </div>
375. </div>
376. </div>
377. </div>
378. </div>
379. </div>
380. </div>
381. </div>
382. </div>
383. </div>
384. </div>
385. </div>
386. </div>
387. </div>
388. </div>
389. </div>
390. </div>
391. </div>
392. </div>
393. </div>
394. </div>
395. </div>
396. </div>
397. </div>
398. </div>
399. </div>
400. </div>
401. </div>
402. </div>
403. </div>
404. </div>
405. </div>
406. </div>
407. </div>
408. </div>
409. </div>
410. </div>
411. </div>
412. </div>
413. </div>
414. </div>
415. </div>
416. </div>
417. </div>
418. </div>
419. </div>
420. </div>
421. </div>
422. </div>
423. </div>
424. </div>
425. </div>
426. </div>
427. </div>
428. </div>
429. </div>
430. </div>
431. </div>
432. </div>
433. </div>
434. </div>
435. </div>
436. </div>
437. </div>
438. </div>
439. </div>
440. </div>
441. </div>
442. </div>
443. </div>
444. </div>
445. </div>
446. </div>
447. </div>
448. </div>
449. </div>
450. </div>
451. </div>
452. </div>
453. </div>
454. </div>
455. </div>
456. </div>
457. </div>
458. </div>
459. </div>
460. </div>
461. </div>
462. </div>
463. </div>
464. </div>
465. </div>
466. </div>
467. </div>
468. </div>
469. </div>
470. </div>
471. </div>
472. </div>
473. </div>
474. </div>
475. </div>
476. </div>
477. </div>
478. </div>
479. </div>
480. </div>
481. </div>
482. </div>
483. </div>
484. </div>
485. </div>
486. </div>
487. </div>
488. </div>
489. </div>
490. </div>
491. </div>
492. </div>
493. </div>
494. </div>
495. </div>
496. </div>
497. </div>
498. </div>
499. </div>
500. </div>
501. </div>
502. </div>
503. </div>
504. </div>
505. </div>
506. </div>
507. </div>
508. </div>
509. </div>
510. </div>
511. </div>
512. </div>
513. </div>
514. </div>
515. </div>
516. </div>
517. </div>
518. </div>
519. </div>
520. </div>
521. </div>
522. </div>
523. </div>
524. </div>
525. </div>
526. </div>
527. </div>
528. </div>
529. </div>
530. </div>
531. </div>
532. </div>
533. </div>
534. </div>
535. </div>
536. </div>
537. </div>
538. </div>
539. </div>
540. </div>
541. </div>
542. </div>
543. </div>
544. </div>
545. </div>
546. </div>
547. </div>
548. </div>
549. </div>
550. </div>
551. </div>
552. </div>
553. </div>
554. </div>
555. </div>
556. </div>
557. </div>
558. </div>
559. </div>
560. </div>
561. </div>
562. </div>
563. </div>
564. </div>
565. </div>
566. </div>
567. </div>
568. </div>
569. </div>
570. </div>
571. </div>
572. </div>
573. </div>
574. </div>
575. </div>
576. </div>
577. </div>
578. </div>
579. </div>
580. </div>
581. </div>
582. </div>
583. </div>
584. </div>
585. </div>
586. </div>
587. </div>
588. </div>
589. </div>
590. </div>
591. </div>
592. </div>
593. </div>
594. </div>
595. </div>
596. </div>
597. </div>
598. </div>
599. </div>
600. </div>
601. </div>
602. </div>
603. </div>
604. </div>
605. </div>
606. </div>
607. </div>
608. </div>
609. </div>
610. </div>
611. </div>
612. </div>
613. </div>
614. </div>
615. </div>
616. </div>
617. </div>
618. </div>
619. </div>
620. </div>
621. </div>
622. </div>
623. </div>
624. </div>
625. </div>
626. </div>
627. </div>
628. </div>
629. </div>
630. </div>
631. </div>
632. </div>
633. </div>
634. </div>
635. </div>
636. </div>
637. </div>
638. </div>
639. </div>
640. </div>
641. </div>
642. </div>
643. </div>
644. </div>
645. </div>
646. </div>
647. </div>
648. </div>
649. </div>
650. </div>
651. </div>
652. </div>
653. </div>
654. </div>
655. </div>
656. </div>
657. </div>
658. </div>
659. </div>
660. </div>
661. </div>
662. </div>
663. </div>
664. </div>
665. </div>
666. </div>
667. </div>
668. </div>
669. </div>
670. </div>
671. </div>
672. </div>
673. </div>
674. </div>
675. </div>
676. </div>
677. </div>
678. </div>
679. </div>
680. </div>
681. </div>
682. </div>
683. </div>
684. </div>
685. </div>
686. </div>
687. </div>
688. </div>
689. </div>
690. </div>
691. </div>
692. </div>
693. </div>
694. </div>
695. </div>
696. </div>
697. </div>
698. </div>
699. </div>
700. </div>
701. </div>
702. </div>
703. </div>
704. </div>
705. </div>
706. </div>
707. </div>
708. </div>
709. </div>
710. </div>
711. </div>
712. </div>
713. </div>
714. </div>
715. </div>
716. </div>
717. </div>
718. </div>
719. </div>
720. </div>
721. </div>
722. </div>
723. </div>
724. </div>
725. </div>
726. </div>
727. </div>
728. </div>
729. </div>
730. </div>
731. </div>
732. </div>
733. </div>
734. </div>
735. </div>
736. </div>
737. </div>
738. </div>
739. </div>
740. </div>
741. </div>
742. </div>
743. </div>
744. </div>
745. </div>
746. </div>
747. </div>
748. </div>
749. </div>
750. </div>
751. </div>
752. </div>
753. </div>
754. </div>
755. </div>
756. </div>
757. </div>
758. </div>
759. </div>
760. </div>
761. </div>
762. </div>
763. </div>
764. </div>
765. </div>
766. </div>
767. </div>
768. </div>
769. </div>
770. </div>
771. </div>
772. </div>
773. </div>
774. </div>
775. </div>
776. </div>
777. </div>
778. </div>
779. </div>
780. </div>
781. </div>
782. </div>
783. </div>
784. </div>
785. </div>
786. </div>
787. </div>
788. </div>
789. </div>
790. </div>
791. </div>
792. </div>
793. </div>
794. </div>
795. </div>
796. </div>
797. </div>
798. </div>
799. </div>
800. </div>
801. </div>
802. </div>
803. </div>
804. </div>
805. </div>
806. </div>
807. </div>
808. </div>
809. </div>
810. </div>
811. </div>
812. </div>
813. </div>
814. </div>
815. </div>
816. </div>
817. </div>
818. </div>
819. </div>
820. </div>
821. </div>
822. </div>
823. </div>
824. </div>
825. </div>
826. </div>
827. </div>
828. </div>
829. </div>
830. </div>
831. </div>
832. </div>
833. </div>
834. </div>
835. </div>
836. </div>
837. </div>
838. </div>
839. </div>
840. </div>
841. </div>
842. </div>
843. </div>
844. </div>
845. </div>
846. </div>
847. </div>
848. </div>
849. </div>
850. </div>
851. </div>
852. </div>
853. </div>
854. </div>
855. </div>
856. </div>
857. </div>
858. </div>
859. </div>
860. </div>
861. </div>
862. </div>
863. </div>
864. </div>
865. </div>
866. </div>
867. </div>
868. </div>
869. </div>
870. </div>
871. </div>
872. </div>
873. </div>
874. </div>
875. </div>
876. </div>
877. </div>
878. </div>
879. </div>
880. </div>
881. </div>
882. </div>
883. </div>
884. </div>
885. </div>
886. </div>
887. </div>
888. </div>
889. </div>
890. </div>
891. </div>
892. </div>
893. </div>
894. </div>
895. </div>
896. </div>
897. </div>
898. </div>
899. </div>
900. </div>
901. </div>
902. </div>
903. </div>
904. </div>
905. </div>
906. </div>
907. </div>
908. </div>
909. </div>
910. </div>
911. </div>
912. </div>
913. </div>
914. </div>
915. </div>
916. </div>
917. </div>
918. </div>
919. </div>
920. </div>
921. </div>
922. </div>
923. </div>
924. </div>
925. </div>
926. </div>
927. </div>
928. </div>
929. </div>
930. </div>
931. </div>
932. </div>
933. </div>
934. </div>
935. </div>
936. </div>
937. </div>
938. </div>
939. </div>
940. </div>
941. </div>
942. </div>
943. </div>
944. </div>
945. </div>
946. </div>
947. </div>
948. </div>
949. </div>
950. </div>
951. </div>
952. </div>
953. </div>
954. </div>
955. </div>
956. </div>
957. </div>
958. </div>
959. </div>
960. </div>
961. </div>
962. </div>
963. </div>
964. </div>
965. </div>
966. </div>
967. </div>
968. </div>
969. </div>
970. </div>
971. </div>
972. </div>
973. </div>
974. </div>
975. </div>
976. </div>
977. </div>
978. </div>
979. </div>
980. </div>
981. </div>
982. </div>
983. </div>
984. </div>
985. </div>
986. </div>
987. </div>
988. </div>
989. </div>
990. </div>
991. </div>
992. </div>
993. </div>
994. </div>
995. </div>
996. </div>
997. </div>
998. </div>
999. </div>
1000. </div>
```

```

84.     }
85.     </script>
86.     <select id="setStatus"></select>
87. </body>
88. </html>

```

Form Validation

Real- or near-realtime input validation has always been among the more difficult features in web application development, especially when the check requires database access. Jaxer's ability to mix between client and server execution, with the security added by minimizing over the wire database interaction, removes most of the obstacles.

```

01. <html>
02. <body>
03.     <script runat="server-proxy">
04.         function masterValidations() {
05.             Jaxer.session['ErrorMessage'] = new Array;
06.
07.             // other validations
08.             employeeUsernameValid(empUsername);
09.             // other validations
10.
11.             if(Jaxer.session['ErrorMessage'].length > 0) {
12.                 /* handle the accumulated errors
13.                  * return false to block
14.                  */
15.                 return false;
16.             } else {
17.                 return true;
18.             }
19.         }
20.
21.         function employeeUsernameValid(empName) {
22.             /* Validating an employee username has multiple rules:
23.              *
24.              * - Must be unique (database test)
25.              * - Must be between 10 and 25 characters (length test)
26.              * - Must contain only alphabetical characters or underscore (grep test)
27.              *
28.              * Return false at the first violation, skipping the remainder
29.              */
30.
31.             /* Since this is likely to be one of many validations,
32.              * how the error message is used is outside the scope of this example.
33.              * Here we just add it to the session ErrorMessage hash
34.              */
35.
36.             if(empName.length > 9 && empName.length < 26) {
37.                 Jaxer.session['ErrorMessage'] << 'Username must be between 10 and 25
38.                 ↵ characters long';
39.                 return false;
40.             }
41.             if(!Jaxer.Util.String.grep(empName, '/^\w+$/')) {
42.                 Jaxer.session['ErrorMessage'] << 'Username must contain only alphabetical
43.                 ↵ characters or underscore';
44.                 return false;
45.             }
46.             var empNames = Jaxer.DB.execute("SELECT username FROM Employees WHERE
47.             ↵ username="+empName);
48.             if (empNames.rows > 0) {
49.                 Jaxer.session['ErrorMessage'] << 'Username must be unique';
50.                 return false;
51.             }
52.         }
53.     </script>
54.     <label for="empUsername">Employee's computer username</label>
55.     <input type="text" size="20" id="empUsername">
56.     <input type="button" value="Submit" onclick="masterValidations()">
57. </body>
58. </html>

```

Writing to the DB

Adding a Row or Rows

```

01. <script runat="server-proxy">
02.     // Add one new row to a table
03.     // A record in this example has three fields
04.     function addMyItem(aItem) {
05.         var rs;
06.         rs = Jaxer.DB.execute('INSERT into MyItems values(?, ?, ?)', aItem[1], aItem[2],
07.         ↵ aItem[3]);
08.         if(!rs) {
09.             alert('Could not insert new Foo');
10.         }
11.         return rs;
12.     }
13.
14.     // Add mutiple new rows to a table
15.     // A record in this example has three fields
16.     // So the parameter is an array of arrays:
17.     // aSetOfItems = [[val1, val2, val3], [val4, val5, val6], [val7, val8, val9]];
18.     function addMyItems(aSetOfItems) {
19.         var rs = new Array;
20.         rs = Jaxer.DB.mapExecute('INSERT into MyItems values(?, ?, ?)', aSetOfItems)
21.         if(Jaxer.filterInPlace(rs, function(itm, idx) {return itm > 0}).length > 0) {
22.             alert('Could not insert new Items');
23.         }
24.         return rs;
25.     }
26. </script>

```

```
24. }
25. </script>
```

Updating a Row

```
01. <script runat="server-proxy">
02. // Update one row in a table
03. // A record in this example has three fields
04. // fooID parameter is the primary key value of the row to update
05. function updateFoo(aFoo, fooID) {
06.     var rs;
07.     rs = Jaxer.DB.execute('UPDATE Foo SET fn1=?, fn2=?, fn3=? WHERE id=?', [aFoo[1],
08.     aFoo[2], aFoo[3], fooID]);
09.     if(!rs) {
10.         alert('Could not update Foo '+aFoo['id']);
11.     }
12.     return rs;
13. }
14. </script>
```

Deleting a Row

```
01. <script runat="server-proxy">
02. // Delete one row in a table
03. // fooID parameter is the primary key value of the row to update
04. function deleteFoo(fooID) {
05.     var rs;
06.     rs = Jaxer.DB.execute('DELETE Foo WHERE id=?', fooID);
07.     if(!rs) {
08.         alert('Could not delete Foo '+fooID);
09.     }
10.     return rs;
11. }
12. </script>
```